

Build an enterprise-grade AI-powered Audit Evidence Review. The purpose of this application is to make Agentic AI observability, traceability, and ROI visible to business leaders who may or may not come from Tech Domain.

This is NOT a generic dashboard. The **observability layer is the primary differentiator**. Every LLM call must be instrumented. Every agent step must be traceable. Every dollar of token spend must be visible. Every audit finding must be defensible with a reasoning log.

=====

TECH STACK

=====

- Frontend: React (Vite) + Tailwind CSS + shadcn/ui + Recharts
- Backend: Node.js + Express
- Database: PostgreSQL with Drizzle ORM (Replit default)
- AI: OpenAI API directly (model: gpt-4o; fallback: gpt-4o-mini)
- File parsing: pdf-parse, mammoth (docx), xlsx (sheetjs), sharp (images)
- State: TanStack Query for server state
- Routing: wouter

=====

AGENT ARCHITECTURE (this is mandatory — do not collapse into one LLM call)

=====

Implement a 4-agent pipeline. This is going to be a multi-agent system. Each agent is a separate LLM call with its own system prompt, its own model, its own span in the trace.

Agent 1 — EXTRACTOR

Input: raw text from uploaded document

Job: extract structured entities (vendor, invoice_number, amount, currency, date, PO_reference, approver_name, contract_id, payment_terms)

Output: JSON

Agent 2 — CONTROL VALIDATOR

Input: extracted entities + audit control rules

Job: check each of the 5 controls (missing approval, missing signature, threshold breach, expired contract, duplicate

invoice) and return violation list with evidence

Output: JSON with control_id, violated (bool), evidence_quote, confidence

Agent 3 — RISK CLASSIFIER & FINDING DRAFTER

Input: violations from Agent 2

Job: classify each violation as Low/Medium/High/Critical and draft

an audit observation (finding, impact, recommendation) in the

house style of a Big 4 audit report

Output: JSON list of findings

Agent 4 — REFLECTION / CRITIC (this is the Module 2 Reflection pattern)

Input: findings from Agent 3 + original extracted entities

Job: critique the draft findings. Flag any finding that is

(a) not supported by evidence in the document,

(b) over-classified relative to the evidence,

(c) missing a recommendation.

Return the revised, approved findings.

Output: JSON list of approved findings + critic_notes

All four agents run sequentially per document. Each agent's input and output is persisted. Each agent's call to OpenAI is wrapped in an instrumentation layer (see next section).

OBSERVABILITY LAYER (this is the demo's core)

Create a TRACE table in Postgres with the following columns:

trace_id (uuid) — one per document

span_id (uuid) — one per LLM call or tool call

parent_span_id (uuid) — nullable, for the trace tree

document_id (fk)

agent_name (text) — extractor, validator, classifier, critic

span_type (text) — llm_call, tool_call, parse, validation

model (text) — gpt-4o, gpt-4o-mini

input_tokens (int)
output_tokens (int)
total_tokens (int)
cost_usd (numeric(10,6))
latency_ms (int)
status (text) — success, error, timeout, rate_limited
error_message (text, nullable)
started_at (timestamp)
ended_at (timestamp)
request_payload (jsonb) — full prompt sent
response_payload (jsonb) — full response received
reasoning_summary (text) — 1-line human-readable summary of what
this span decided and why

Every OpenAI call must go through a single wrapper function

`callOpenAI(agentName, model, messages, parentSpanId, traceId)` that:

1. Records start time
2. Calls OpenAI
3. Reads usage.prompt_tokens, usage.completion_tokens from response
4. Computes cost from a CONFIGURABLE pricing table:
gpt-4o: \$2.50 input / \$10.00 output per 1M tokens
gpt-4o-mini: \$0.15 input / \$0.60 output per 1M tokens
(store as config so rates can be updated)
5. Inserts a row into the TRACE table
6. Returns the response
7. On error: still inserts a row with status='error' and message

Also instrument NON-LLM steps:

- PDF parsing → span_type='parse'
- Control rule check (deterministic) → span_type='validation'
- Database read/write → span_type='tool_call'

=====

USER INTERFACE — 5 PAGES

=====

PAGE 1: EXECUTIVE OVERVIEW (default landing page)

Top row — 6 KPI cards:

[Documents Analyzed] [Total Findings] [Critical Findings]

[Compliance Score] [Total Tokens] [Total Spend USD/INR]

Middle — Two charts side by side:

Left: Risk distribution (donut: Low/Medium/High/Critical)

Right: Cost-per-document trend (last 20 documents, line chart)

Bottom — "ROI Snapshot" card (this is the money slide):

Documents analyzed today: X

Equivalent auditor hours saved: Y hrs (assume 45 min/doc baseline)

Cost at \$250/hr auditor rate: \$A

Actual AI spend: \$B

Net savings: \$A - \$B = \$C (XX% reduction)

Cost per finding: \$B / num_findings

Display in both USD and INR (use rate 83 INR/USD, configurable)

PAGE 2: UPLOAD & ANALYZE

Drag-drop zone (PDF, DOCX, XLSX, PNG/JPG)

On upload: show a LIVE TRACE PANEL on the right that streams

each agent step as it completes:

[✓] Extractor gpt-4o 1,240 tok \$0.011 1,240 ms

[✓] Validator gpt-4o 890 tok \$0.008 980 ms

[🔄] Classifier gpt-4o running...

[] Critic pending

This is the LIVE OBSERVABILITY VIEW. It must feel like watching the agent think. Use server-sent events or polling every 500ms.

PAGE 3: TRACE EXPLORER (the Module 3 centerpiece)

Left pane: list of all documents analyzed

Right pane when a document is selected:

- a) Waterfall view of all spans for that document's trace
(horizontal bar chart, x-axis = time, each bar = one span,
colored by agent, hover shows tokens/cost/latency)
- b) Expandable span tree below the waterfall:
 - ▼ Extractor (gpt-4o, 1,240 tok, \$0.011, 1,240 ms)
Input: [show first 300 chars of prompt]
Output: [show extracted JSON]
Reasoning: "Identified vendor 'Acme Corp' and invoice
amount \$87,500 above approval threshold."
 - ▼ Validator (gpt-4o, 890 tok, \$0.008, 980 ms)
...
 - ▼ Classifier ...
 - ▼ Critic ...
Critic notes: "Finding 2 reclassified Medium→High based
on missing director approval per policy 4.2."
- c) "Why this finding?" button on every finding → opens the
exact span chain that produced it. THIS IS THE
DEFENSIBILITY DEMO.

PAGE 4: COST & PERFORMANCE ANALYTICS

Charts:

- Total spend over time (USD & INR toggle)
- Token consumption by agent (stacked bar)
- Latency distribution: p50, p95, p99 per agent
- Model mix (gpt-4o vs gpt-4o-mini usage)
- Cost per finding by risk category

Table:

Top 10 most expensive document analyses with drill-down to trace

PAGE 5: GOVERNANCE & FAILURE MODES (Module 3 explicit content)

Three tabs:

Tab A — Human-in-the-Loop Queue:

Findings flagged for review because:

- Critic agent disagreed with Classifier
- Confidence below 0.7
- Critical risk classification

Each row: document, finding, who-flagged, action buttons

(Approve / Reject / Reassign)

Tab B — Failure Mode Anatomy:

List of failed spans in last 7 days, grouped by failure type:

- OpenAI rate limit (with count, last occurrence)
- JSON parse failure (model returned non-JSON)
- Document parse failure (corrupt PDF, password-protected)
- Reflection caught hallucination (critic rejected finding)

Each group expandable to show traces.

Tab C — Immutable Audit Log:

Every user action (upload, analyze, approve, reject, export)

timestamped, user-attributed, never editable.

=====

PREDEFINED CONTROL RULES (deterministic)

=====

The Validator agent uses the LLM to extract intent but the final control violation is computed deterministically (this is the Module 1 lesson: precision-critical work uses rules, not GenAI).

Hardcode these 5 rules:

R1 Missing approval: invoice amount > 50,000 AND approver is null

R2 Missing signature: document type = invoice/contract AND signature_present = false

R3 Threshold breach: invoice amount > vendor_contract_max_value

R4 Expired contract: invoice date > contract_end_date

R5 Duplicate invoice: invoice_number already exists in DB for same vendor

Each rule has a fixed default risk weight (configurable in admin UI).

DEMO DATA

Pre-seed the database with 8 sample documents that demonstrate each failure mode at least once:

1. Clean invoice (no findings — shows the system doesn't over-flag)
2. Invoice above threshold, no approver → R1 violation, HIGH risk
3. Contract expired 30 days before invoice → R4 violation, MEDIUM
4. Duplicate invoice → R5 violation, CRITICAL
5. Invoice with missing signature → R2 violation, MEDIUM
6. Threshold breach with chained approvals → LOW risk (controlled)
7. PDF parse failure (corrupt file) — demonstrates failure mode panel
8. Document where Critic overrules Classifier — demonstrates Reflection

Add a "Run Demo Sequence" button that processes all 8 documents sequentially with a 2-second pause between each, so the live trace panel can be projected during the live demo.

BUSINESS-LEADER PRESENTATION MODE

Add a top-right toggle: "Presentation Mode"

When enabled:

- Hides developer details (raw JSON, technical span IDs)
- Enlarges KPI numbers and ROI panel
- Adds presenter notes overlay explaining each panel in plain English ("This panel shows every step the AI agent took.

Click any step to see the reasoning.")

- Replaces small charts with larger versions optimized for projection (1080p screen distance)

STYLING

- Corporate, restrained palette: navy primary (#0B2545), white, slate greys, with risk badges in standard colors
(Critical=#B91C1C, High=#EA580C, Medium=#CA8A04, Low=#16A34A)
- Typography: Inter or system-ui, no decorative fonts
- Wide whitespace, dense data tables only on Trace Explorer page
- Avoid any "consumer app" visual cues (no gradients, no emojis in production UI, no playful illustrations)

EXPORTS

- Per-document: Export audit findings as PDF (Big 4 report format)
- Per-document: Export full trace as JSON (for regulator review)
- Dashboard: Export ROI snapshot as PNG (for inclusion in decks)
- All exports include the trace_id for traceability

ENVIRONMENT VARIABLES

OPENAI_API_KEY — required

OPENAI_BASE_URL — optional, defaults to api.openai.com

DEFAULT_MODEL — defaults to gpt-4o

AUDITOR_HOURLY_USD — defaults to 250 (for ROI calc)

INR_USD_RATE — defaults to 83

BASELINE_MIN_PER_DOC — defaults to 45 (manual review baseline)

DELIVERABLES

- Working application deployed on Replit
- Seeded with 8 demo documents
- Presentation Mode toggle functional
- README.md with: setup, env vars, demo flow script,
and a "what to say at each step" speaker guide
- Folder structure: clean separation of agents/, observability/,
routes/, components/, db/

Enhancement Prompt 1 — deepen the trace (run after the base build is stable)

Enhance the Trace Explorer page with the following additions:

1. Span-level token breakdown: for every span, show a small horizontal bar visualising input_tokens vs output_tokens with the system prompt portion separated from the user message portion of input.
2. "What did the agent see?" toggle: on each span, allow the reviewer to view the EXACT context window sent to the model, with retrieval chunks highlighted if any.
3. Replay capability: a "Re-run this span with different model" button — re-runs the same prompt against gpt-4o-mini and shows a side-by-side comparison of output, tokens, cost, and latency. This demonstrates the Module 3 message: not every step needs the most expensive model.
4. Cost attribution: pie chart showing what fraction of a document's total cost came from each of the 4 agents. This helps leaders identify which agent to optimize.

Enhancement Prompt 2 — sharpen the ROI

Add a "Business Case" page targeted at executive leadership.

1. Interactive ROI calculator with three sliders:
 - Documents per month (default 5,000)
 - Average minutes per manual review (default 45)

- Auditor blended hourly rate (default \$250)

Output: monthly savings, annual savings, payback period assuming a \$50K platform investment.

2. Side-by-side comparison table:

	Manual	Agentic	Delta
Avg time per doc	45 min	18 sec	-99.3%
Coverage	8%	100%	+12.5x
Reviewer variance	±18%	0%	eliminated
Cost per doc	\$187.50	\$0.04	-99.98%
Traceability	Partial	Full	new capability

3. Risk-adjusted view: factor in the new risks that agentic AI introduces (hallucination, model drift, data leakage) and show how the observability layer mitigates each. Each risk gets a column: Risk / Likelihood / Mitigation / Residual Risk / Where to Verify (link to the relevant observability feature).

4. Export the entire Business Case page as a 1-page PDF that a partner can drop into a client deck.